

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЧЕРКАСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ БОГДАНА ХМЕЛЬНИЦЬКОГО
Факультет обчислювальної техніки, інтелектуальних та управляючих систем
Кафедра інформаційних технологій

Звіт
з лабораторної роботи №4
Тема: Програмне моделювання машинних алгоритмів додавання
та віднімання чисел з плаваючою крапкою
Варіант: 7

Виконала:

студентка групи КС-242-1 Гук Марія Олегівна

Перевірив:

ст. викладач кафедри ПЗАС Мисник Б.В.

ЗАВДАННЯ

Програмно промоделювати машинний алгоритм віднімання чисел з плаваючою крапкою. Перший операнд представити у форматі 16 розрядів, другий – у форматі 8 розрядів.

ЛІСТИНГ ПРОГРАМИ

[GitHub](#)

```
using System;
using System.Diagnostics.CodeAnalysis;
using System.Reflection.Metadata;
using System.Runtime.CompilerServices;
using System.Text;
using System.Text.RegularExpressions;

namespace Lab4
{
    struct Float
    {
        public string sign;
        public string mantissa;
        public string exp;

        public Float(string sign, string mantissa, string
exp)
        {
            this.sign = sign;
            this.mantissa = mantissa;
            this.exp = exp;
        }

        public Float(string str)
```

```

        {
            sign = str[0] == '-' ? "1" : "0";
            str = str.Split(',', '.')[1];
            var parts = str.Split('*').Select(x =>
x.Trim()).ToArray();
            mantissa = parts[0];
            exp = parts[1].Split('^').Select(x =>
x.Trim()).ToArray()[1];
        }
        public void InvertSign() => sign = sign == "1" ?
"0" : "1";
        public void ShiftMantissaRight(int shift)
        {
            mantissa = new string('0', shift) + mantissa;
            mantissa = mantissa[..(mantissa.Length -
shift)];
        }
        public void ShiftMantissaLeft(int shift)
        {
            mantissa = mantissa[shift..] + new
string('0', shift);
        }
        public void AdditionalSwap()
        {
            if (sign == "1")
            {
                char[] additionalCode =
mantissa.ToCharArray();
                bool foundOne = false;

```

```

        for (int i = additionalCode.Length - 1; i
    >= 0; i--)
    {
        if (foundOne)
        {
            additionalCode[i] =
additionalCode[i] == '0' ? '1' : '0';
        }
        if (additionalCode[i] == '1' &&
!foundOne)
        {
            foundOne = true;
        }
    }
    mantissa = new string(additionalCode);
}

public static Float operator +(Float a, Float b)
{
    int maxLength = Math.Max(a.mantissa.Length,
b.mantissa.Length);
    a.mantissa = a.mantissa.PadLeft(maxLength,
'0');
    b.mantissa = b.mantissa.PadLeft(maxLength,
'0');

    char carry = '0';
    char[] result = new char[maxLength + 1];

    for (int i = maxLength - 1; i >= 0; i--)

```

```

        {
            int sum = (a.mantissa[i] - '0') +
(b.mantissa[i] - '0') + (carry - '0');
            if (sum >= 2)
            {
                carry = '1';
                sum -= 2;
            }
            else
            {
                carry = '0';
            }
            result[i + 1] = (sum == 0) ? '0' : '1';
        }
        result[0] = carry;

        string resultSign = a.sign;
        if (a.sign != b.sign)
        {
            resultSign =
a.mantissa.CompareTo(b.mantissa) >= 0 ? a.sign : b.sign;
        }
        else if (carry == '1' && resultSign == "1")
        {
            resultSign = "10";
        }
        else if (carry == '1' && resultSign == "0")
        {
            resultSign = "1";
        }
    }

```

```

        return new(resultSign, new
string(result).TrimStart('0'), a.exp);
    }
    public void CheckNormalization()
    {
        if (sign.Length > 1)
        {
            ShiftMantissaRight(1);
            exp = (Convert.ToInt32(exp, 2) +
1).BitLook();
            sign = "1";
        }
        else if (sign[0] == mantissa[0])
        {
            int cnt = 0;
            do
            {
                cnt++;
                ShiftMantissaLeft(1);
                exp = (Convert.ToInt32(exp, 2) -
1).BitLook();
            }
            while (cnt<15 && (sign=="1"? mantissa[0]
!= '0': mantissa[0] != '1'));
        }
    }
    public string Result()
    {
        string res = "";

```

```

        if (sign == "1") res = "-";
        res += "0," + mantissa + " * 2^" + exp;
        return res;
    }

    public override string ToString() =>
$"{sign}|{mantissa}|{exp}";
    }

    static class Program
    {
        public static string BitLook(this int a)
        {
            StringBuilder sb = new();
            foreach (var i in Enumerable.Range(0,
15).Reverse())
                sb.Append((a & 0x1 << i) == 0b0 ? "0" :
"1");

            return sb.ToString().TrimStart('0');
        }

        static void Main()
        {
            Console.InputEncoding = Encoding.Unicode;
            Console.OutputEncoding = Encoding.Unicode;
            //Ввід
            Input(out string data1, out string data2);
            Float num1 = new(data1);
            Float num2 = new(data2);
            num2.InvertSign();

```

```
//Перевід в прямий код
Console.WriteLine("КРОК 0 (Запис в прямому
кодi):");

Console.WriteLine($"Перше число в прямому
кодi: {num1}");

Console.WriteLine($"Друге число в прямому
кодi: {num2} (Зміна знаку для виконання віднімання)");


Console.WriteLine("КРОК 1 (Урівноваження
порядків доданків):");

int diff =
Math.Abs(Convert.ToInt32(num1.exp, 2) -
Convert.ToInt32(num2.exp, 2));

if (Convert.ToInt32(num1.exp, 2) >
Convert.ToInt32(num2.exp, 2))
    num2.ShiftMantissaRight(diff);
else num1.ShiftMantissaRight(diff);

string max =
Math.Max(Convert.ToInt32(num1.exp, 2),
Convert.ToInt32(num2.exp, 2)).BitLook();

num1.exp = max;
num2.exp = max;

Console.WriteLine($"Перше число після
урівноваження порядків: {num1}");

Console.WriteLine($"Друге число після
урівноваження порядків: {num2}");
```



```
        Console.WriteLine("КРОК 2 (Перетворення  
мантис чисел в додатковий код):");  
        num1.AdditionalSwap();  
        num2.AdditionalSwap();  
        Console.WriteLine($"Перше число в додатковому  
кодi: {num1}");  
        Console.WriteLine($"Друге число в додатковому  
кодi: {num2}");
```

```
        Console.WriteLine("КРОК 3 (Додавання  
мантис):");  
        Float sum = num1 + num2;  
        Console.WriteLine($"Сума цих чисел в  
додатковому кодi: {sum}");
```

```
        Console.WriteLine("КРОК 4 (Денормалізація  
результату):");  
        sum.CheckNormalization();  
        Console.WriteLine($"Результат після  
нормалізування: {sum}");
```

```
        Console.WriteLine("КРОК 5 (Подання  
результату):");
```

```

        sum.AdditionalSwap();
        Console.WriteLine($"Результат в прямому коді:
{sum}");
        Console.WriteLine($"Результат:
{sum.Result()}");
    }

```

```

        private static void Input(out string num1, out
string num2)
        {
            do
            {
                Console.WriteLine("Введіть перше бінарне
число в такому вигляді: -0,001010101001111 * 2^10 (16
знаків).");
                num1 = Console.ReadLine();
                string pattern = @"^[-
]?0[,\.][01]{15}\s*\s2\^[01]+$";
                if (Regex.IsMatch(num1, pattern)) break;
                else Console.WriteLine("Помилка!
Спробуйте ще раз.");
            }
            while (true);
            do
            {
                Console.WriteLine("Введіть друге бінарне
число в такому вигляді: 0,0011101 * 2^01 (8 знаків).");
                num2 = Console.ReadLine();
                string pattern = @"^[-
]?0[,\.][01]{7}\s*\s2\^[01]+$";

```

```
        if (Regex.IsMatch(num2, pattern)) break;
        else Console.WriteLine("Помилка!
Спробуйте ще раз.");
    }
    while (true);
}
}
```

РЕЗУЛЬТАТ РОБОТИ ПРОГРАМИ

1. Виконання програми (Рис. 1.1) та (Рис. 1.2):

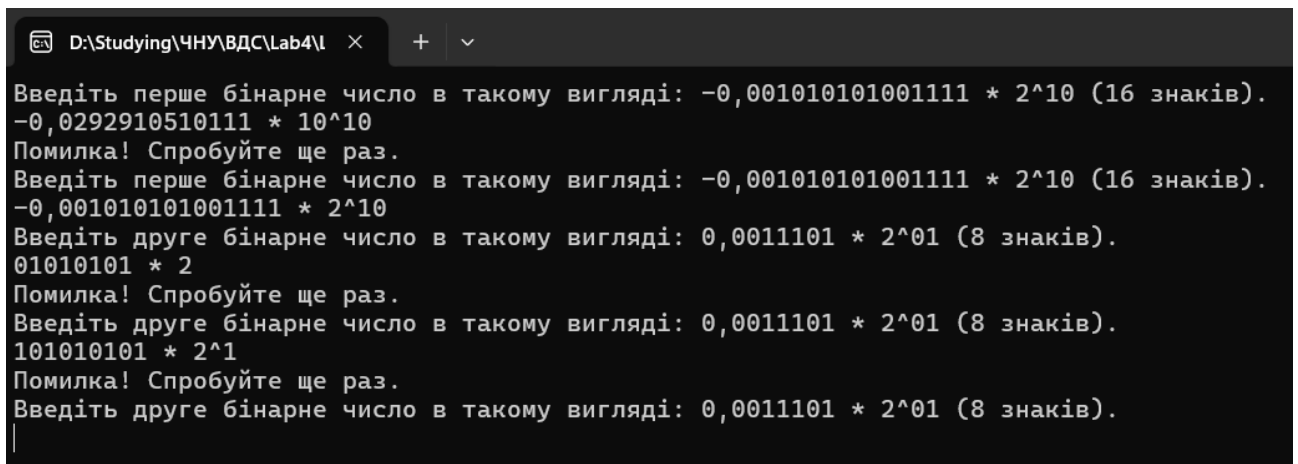
```
Microsoft Visual Studio Debug Console
Введіть перше бінарне число в такому вигляді: -0,001010101001111 * 2^10 (16 знаків).
0,001010101001111 * 2^10
Введіть друге бінарне число в такому вигляді: 0,0011101 * 2^01 (8 знаків).
-0,0011101 * 2^01
КРОК 0 (Запис в прямому коді):
Перше число в прямому коді: 0|001010101001111|10
Друге число в прямому коді: 0|0011101|01 (Зміна знаку для виконання віднімання)
КРОК 1 (Урівноваження порядків доданків):
Перше число після урівноваження порядків: 0|001010101001111|10
Друге число після урівноваження порядків: 0|0001110|10
КРОК 2 (Перетворення мантий чисел в додатковий код):
Перше число в додатковому коді: 0|001010101001111|10
Друге число в додатковому коді: 0|0001110|10
КРОК 3 (Додавання мантий):
Сума цих чисел в додатковому коді: 0|1010101011101|10
КРОК 4 (Денормалізація результату):
Результат після нормалізування: 0|1010101011101|10
КРОК 5 (Подання результату):
Результат в прямому коді: 0|1010101011101|10
Результат: 0,1010101011101 * 2^10
```

Рис 1.1. Приклад виконання програми в консолі.

```
Microsoft Visual Studio Debug Console
Введіть перше бінарне число в такому вигляді: -0,001010101001111 * 2^10 (16 знаків).
-0,000000101001000 * 2^111
Введіть друге бінарне число в такому вигляді: 0,0011101 * 2^01 (8 знаків).
-0,0000100 * 2^101
КРОК 0 (Запис в прямому коді):
Перше число в прямому коді: 1|000000101001000|111
Друге число в прямому коді: 0|0000100|101 (Зміна знаку для виконання віднімання)
КРОК 1 (Урівноваження порядків доданків):
Перше число після урівноваження порядків: 1|000000101001000|111
Друге число після урівноваження порядків: 0|0000001|111
КРОК 2 (Перетворення мантий чисел в додатковий код):
Перше число в додатковому коді: 1|111111010111000|111
Друге число в додатковому коді: 0|0000001|111
КРОК 3 (Додавання мантий):
Сума цих чисел в додатковому коді: 1|111111010111001|111
КРОК 4 (Денормалізація результату):
Результат після нормалізування: 1|010111001000000|1
КРОК 5 (Подання результату):
Результат в прямому коді: 1|101000111000000|1
Результат: -0,101000111000000 * 2^1
```

Рис 1.2. Приклад виконання програми в консолі.

2. Виконання програми при помилках у введенні (Рис. 1.3);



```
D:\Studying\ЧНУ\ВДС\Lab4\l  × + v
Введіть перше бінарне число в такому вигляді: -0,001010101001111 * 2^10 (16 знаків).
-0,0292910510111 * 10^10
Помилка! Спробуйте ще раз.
Введіть перше бінарне число в такому вигляді: -0,001010101001111 * 2^10 (16 знаків).
-0,001010101001111 * 2^10
Введіть друге бінарне число в такому вигляді: 0,0011101 * 2^01 (8 знаків).
01010101 * 2
Помилка! Спробуйте ще раз.
Введіть друге бінарне число в такому вигляді: 0,0011101 * 2^01 (8 знаків).
101010101 * 2^1
Помилка! Спробуйте ще раз.
Введіть друге бінарне число в такому вигляді: 0,0011101 * 2^01 (8 знаків).
|
```

Рис. 1.3. Приклад виконання програми в консолі при помилках у введенні.

ВИСНОВОК

У ході виконання лабораторної роботи №4 було програмно змодельовано машинний алгоритм віднімання чисел у форматі з плаваючою крапкою. Було реалізовано операцію для операндів різної розрядності: 16 біт для першого та 8 біт для другого.

Реалізована програма успішно виконує віднімання, що включає етапи вирівнювання порядків, перетворення мантис у відповідний код для додавання, безпосереднє додавання мантис та нормалізацію отриманого результату. Програма також коректно обробляє введені дані, у тому числі випадки помилкового введення, що було продемонстровано під час тестування.

Отримані результати підтвердили правильність реалізації змодельованого алгоритму та основних кроків обробки чисел з плаваючою крапкою. Виконана робота сприяла глибшому розумінню процесів машинної арифметики з плаваючою крапкою, особливостей роботи з різними форматами представлення чисел, та їхнього програмного моделювання.